

Deep learning foundations for NLP

1.1 From biological neurons to the artificial perceptron

Having established the probabilistic formulation of language modeling and the combinatorial limitations of discrete symbol counting in Chapter ??, we now develop the neural computational foundations that allow language models to scale. In contemporary Natural Language Processing and AI-assisted Software Engineering, next-token prediction is reformulated as a high-dimensional, non-linear classification problem over continuous vector spaces. Rather than storing astronomical tables of discrete counts, we parameterize language predictors as continuous functions whose weights are optimized via gradient descent.

The foundational building block of modern deep learning is the *perceptron*, originally inspired by biological neurons in the human nervous system. A biological neuron receives electrochemical signals from upstream cells through branching *dendrites*, integrates these incoming potentials in its central cell body (*soma*), and, when the accumulated electrical voltage exceeds a physiological threshold (the action potential), fires a spike through its *axon* to downstream synapses.

In computational neural networks, this biological mechanism was first formalized by Warren McCulloch and Walter Pitts in 1943 [?] as a threshold logic unit and subsequently refined by Frank Rosenblatt in 1958 [?] into the trainable *Perceptron*.

Definition 1.1: The artificial perceptron

Let $\mathbf{x} = [x_1, x_2, \dots, x_n]^\top \in \mathbb{R}^n$ represent an input feature vector, and let $\mathbf{w} = [w_1, w_2, \dots, w_n]^\top \in \mathbb{R}^n$ represent the corresponding parameter weights, with scalar bias $w_0 = b \in \mathbb{R}$. The artificial perceptron computes:

$$y = f \left(w_0 + \sum_{i=1}^n w_i x_i \right) = f (w_0 + w_1 x_1 + w_2 x_2 + \dots + w_n x_n), \quad (1.1)$$

where $f : \mathbb{R} \rightarrow \mathbb{R}$ is the activation function. By adopting the standard convention of augmenting the input vector with a constant dummy feature $x_0 = 1$ and defining the parameter vector as $\mathbf{w} = [w_0, w_1, \dots, w_n]^\top \in \mathbb{R}^{n+1}$, Equation (??) is expressed in compact vector inner-product notation:

$$y = f \left(\sum_{i=0}^n w_i x_i \right) = f (\mathbf{w}^\top \mathbf{x}) \quad \text{with } x_0 = 1, w_0 = \text{bias}. \quad (1.2)$$

Figure ?? illustrates the structural mechanics of the artificial perceptron: input signals are weighted by synaptic connection strengths, summed at a central junction, and passed through an activation function to generate the output signal.

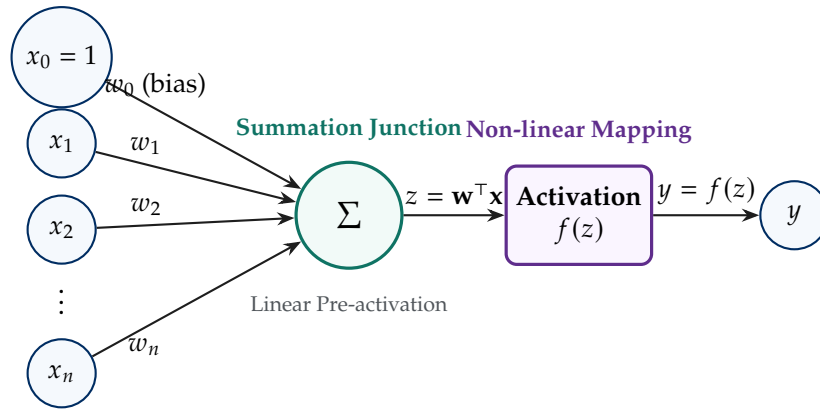


Figure 1.1: Architectural schematic of the artificial perceptron. Inputs x_i are modulated by synaptic weights w_i , accumulated with bias w_0 at the summation junction Σ to yield pre-activation $z = \mathbf{w}^T \mathbf{x}$, and mapped through activation function $f(z)$ to produce scalar output y .

1.1.1 The perceptron in two dimensions: linear decision hyperplane

To establish geometric intuition, consider a two-dimensional feature space $\mathbf{x} = [x_1, x_2]^T \in \mathbb{R}^2$ with weights $\mathbf{w} = [w_1, w_2]^T$ and bias w_0 . The pre-activation value is:

$$z(\mathbf{x}) = w_1 x_1 + w_2 x_2 + w_0. \quad (1.3)$$

When the perceptron acts as a binary classifier using a step or sign activation ($y = \text{sgn}(z)$), the decision boundary separating the two output classes is precisely the set of points where the pre-activation vanishes:

$$\mathcal{H} = \{\mathbf{x} \in \mathbb{R}^2 \mid w_1 x_1 + w_2 x_2 + w_0 = 0\}. \quad (1.4)$$

Geometric interpretation of weights and bias. In two dimensions, Equation (??) defines a straight line in the (x_1, x_2) Cartesian plane:

- **Normal Vector:** The weight vector $\mathbf{w} = [w_1, w_2]^T$ is strictly perpendicular (orthogonal) to the separating line $\mathcal{H}$. For any two points $\mathbf{x}_a, \mathbf{x}_b \in \mathcal{H}$:

$$\mathbf{w}^T \mathbf{x}_a + w_0 = 0 \quad \text{and} \quad \mathbf{w}^T \mathbf{x}_b + w_0 = 0 \implies \mathbf{w}^T (\mathbf{x}_a - \mathbf{x}_b) = 0.$$

The vector $\mathbf{w}$ points directly into the positive decision region ($z > 0$).

- **Orthogonal Distance to Origin:** The shortest Euclidean distance from the origin $\mathbf{0} = [0, 0]^T$ to the separating hyperplane $\mathcal{H}$ is:

$$d = \frac{|w_0|}{\|\mathbf{w}\|_2} = \frac{|w_0|}{\sqrt{w_1^2 + w_2^2}}. \quad (1.5)$$

The bias w_0 shifts the hyperplane parallel to itself: if $w_0 = 0$, the decision boundary passes exactly through the origin; adjusting w_0 translates the boundary without altering its orientation.

Figure ?? diagrams this geometric relationship in $\mathbb{R}^2$.

1.1.2 Connection to classical linear and logistic regression

The artificial perceptron unifies two foundational statistical models studied in classical data analysis:

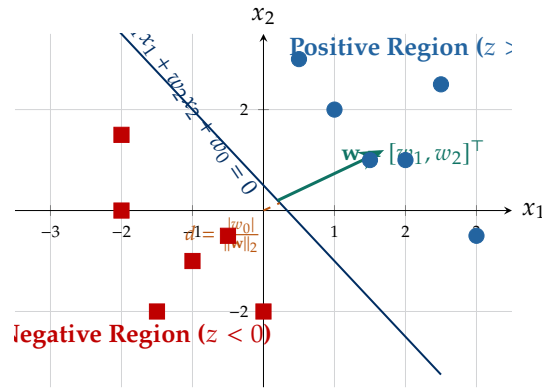


Figure 1.2: Geometric representation of the perceptron decision boundary in $\mathbb{R}^2$. The line $w_1x_1 + w_2x_2 + w_0 = 0$ partitions feature space into positive ($\mathcal{H}^+$) and negative ($\mathcal{H}^-$) half-spaces. The weight vector $\mathbf{w}$ defines the normal vector to the separating hyperplane, and the scalar bias w_0 determines its distance from the origin.

- **Linear Regression:** If the activation function is the linear identity $f(z) = z$, Equation (??) computes:

$$y = \mathbf{w}^\top \mathbf{x},$$

predicting an unbounded real-valued continuous response variable.

- **Logistic Regression:** If the activation function is the logistic sigmoid $f(z) = \sigma(z) = \frac{1}{1+e^{-z}}$, Equation (??) computes:

$$y = \sigma(\mathbf{w}^\top \mathbf{x}) = \mathbb{P}(Y = 1 \mid \mathbf{x}),$$

predicting the posterior Bernoulli probability of belonging to the positive class.

1.1.3 The fatal barrier of single-layer perceptrons: XOR linear inseparability

Despite its mathematical elegance, the single-layer perceptron suffers from a fatal representational barrier that brought artificial intelligence research to a virtual standstill for over a decade. In their seminal 1969 monograph “*Perceptrons*”, Marvin Minsky and Seymour Papert demonstrated that a single-layer perceptron cannot compute the basic *exclusive-or* (XOR) logical function.

The XOR truth table and linear system. Consider the XOR logic function over binary inputs $x_1, x_2 \in \{0, 1\}$:

x_1	x_2	Target $y = x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

For a single-layer perceptron with weights w_1, w_2 and bias w_0 to compute this function, there must exist parameters such that the linear boundary $w_1x_1 + w_2x_2 + w_0$ outputs positive values ($z \geq 0$) for target 1 and negative values ($z < 0$) for target 0. Substituting the four input pairs yields the system of linear inequalities:

$$(x_1 = 0, x_2 = 0) \implies w_0 < 0, \quad (1.6)$$

$$(x_1 = 1, x_2 = 0) \implies w_1 + w_0 \geq 0, \quad (1.7)$$

$$(x_1 = 0, x_2 = 1) \implies w_2 + w_0 \geq 0, \quad (1.8)$$

$$(x_1 = 1, x_2 = 1) \implies w_1 + w_2 + w_0 < 0. \quad (1.9)$$

Proof by mathematical contradiction. Summing inequalities (??) and (??):

$$(w_1 + w_0) + (w_2 + w_0) \geq 0 \implies w_1 + w_2 + 2w_0 \geq 0 \implies w_1 + w_2 \geq -2w_0. \quad (1.10)$$

Now examine inequality (??):

$$w_1 + w_2 + w_0 < 0 \implies w_1 + w_2 < -w_0. \quad (1.11)$$

Combining inequalities (??) and (??) yields:

$$-2w_0 \leq w_1 + w_2 < -w_0 \implies -2w_0 < -w_0.$$

Adding $2w_0$ to both sides yields:

$$0 < w_0 \implies w_0 > 0.$$

However, inequality (??) strictly demands that $w_0 < 0$. The assumption that a single linear hyperplane can compute XOR produces a direct mathematical contradiction ($w_0 > 0$ and $w_0 < 0$).

■

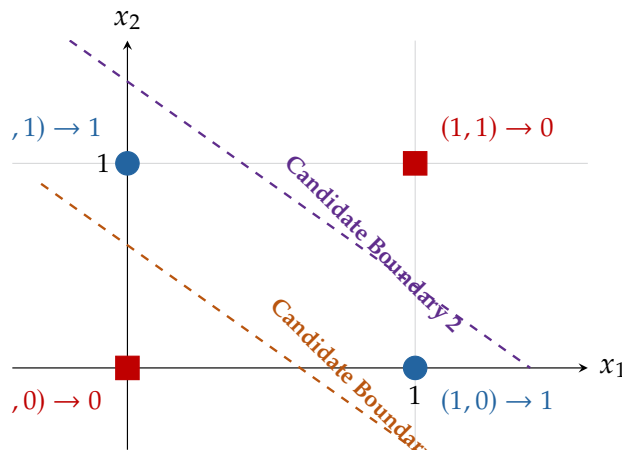


Figure 1.3: The XOR linear inseparability barrier (Minsky & Papert 1969). Class 1 points (0, 1) and (1, 0) (blue circles) cannot be separated from Class 0 points (0, 0) and (1, 1) (red squares) by any single linear hyperplane $w_1x_1 + w_2x_2 + w_0 = 0$. Any single line either isolates (0, 0) or (1, 1), but cannot simultaneously group both off-diagonal points.

As visualized in Figure ??, no straight line can separate the diagonal pairs. Because natural language and software engineering phenomena are riddled with non-linear relationships (e.g., mutually dependent programming idioms, nested conditional scopes), single-layer linear perceptrons are fundamentally inadequate.

The causal question. How can we overcome this linear inseparability barrier? An immediate intuitive engineering hypothesis presents itself: *If one linear perceptron can draw one line, can we solve non-linear problems simply by cascading multiple linear perceptrons into a multi-layer network?* We test this hypothesis next.

1.2 Stacking linear layers and the proof of linear stacking collapse

1.2.1 Fully connected linear layers (dense layers)

In multi-layer architectures, individual perceptrons do not operate in isolation; instead, multiple neurons run concurrently across a shared input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, forming a *dense layer* (or fully connected layer).

Let d_{in} denote the input feature dimensionality and d_{out} denote the number of hidden or output units. Stacking the individual weight vectors $\mathbf{w}_j \in \mathbb{R}^{d_{\text{in}}}$ as columns of a weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and collecting the scalar biases b_j into a bias vector $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$, the layer computation expresses compactly in matrix notation:

$$\mathbf{y} = f(\mathbf{W}^T \mathbf{x} + \mathbf{b}), \quad (1.12)$$

where $f(\cdot)$ is applied elementwise. Figure ?? diagrams this fully connected bipartite structure with explicit tensor dimensions and connection weights.

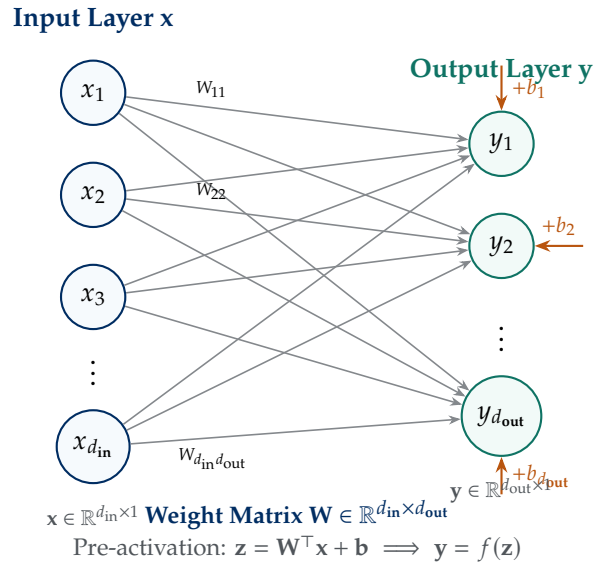


Figure 1.4: Architecture of a fully connected dense layer mapping an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ to output $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$. Each directed edge carries an individual scalar weight W_{ij} , and each output neuron incorporates an additive bias b_j .

Exam insights: Parameter counting across dense layers

A standard oral and written examination question tests exact parameter counting across stacked dense layers. For an affine transformation mapping $\mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_{\text{out}}}$:

$$N_{\text{params}} = \underbrace{d_{\text{in}} \times d_{\text{out}}}_{\text{weight parameters}} + \underbrace{d_{\text{out}}}_{\text{bias parameters}} = d_{\text{out}}(d_{\text{in}} + 1). \quad (1.13)$$

Example Calculation: Consider a two-layer Multi-Layer Perceptron (MLP) mapping an embedding of dimension $d_{\text{in}} = 768$ through a hidden layer $d_{\text{hidden}} = 3072$ to an output projection $d_{\text{out}} = 768$:

1. Layer 1: $3072 \times (768 + 1) = 2,359,296 + 3072 = 2,362,368$ parameters.
2. Layer 2: $768 \times (3072 + 1) = 2,359,296 + 768 = 2,360,064$ parameters.
3. Total Parameters: $2,362,368 + 2,360,064 = 4,722,432$ parameters.

Neglecting bias terms during parameter inventory counts is a frequent source of point deductions.

1.2.2 The mathematical proof of linear stacking collapse

Now we return to our hypothesis: *Can we solve XOR and non-linear problems simply by cascading multiple linear layers without activation functions?*

The following theorem demonstrates that without non-linear activations, stacking arbitrary numbers of linear layers provides zero additional representational power over a single-layer linear model.

Theorem 1.1: Linear stacking collapse

Let $f_1 : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $f_2 : \mathbb{R}^m \rightarrow \mathbb{R}^k$ denote two consecutive fully connected layers operating without non-linear activations ($f(x) = x$). Let $\mathbf{W}_1 \in \mathbb{R}^{n \times m}$ and $\mathbf{b}_1 \in \mathbb{R}^m$ be the parameters of the first layer, and let $\mathbf{W}_2 \in \mathbb{R}^{m \times k}$ and $\mathbf{b}_2 \in \mathbb{R}^k$ be the parameters of the second layer:

$$\mathbf{h} = f_1(\mathbf{x}) = \mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1, \quad (1.14)$$

$$\mathbf{z} = f_2(\mathbf{h}) = \mathbf{W}_2^\top \mathbf{h} + \mathbf{b}_2. \quad (1.15)$$

The composite function $\mathbf{z} = f_2(f_1(\mathbf{x}))$ collapses identically to a single affine transformation:

$$\begin{aligned} \mathbf{z} &= \mathbf{W}_2^\top (\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 \\ &= \mathbf{W}_2^\top \mathbf{W}_1^\top \mathbf{x} + \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2 \\ &= (\mathbf{W}_1 \mathbf{W}_2)^\top \mathbf{x} + (\mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2) \\ &= \mathbf{W}_{\text{net}}^\top \mathbf{x} + \mathbf{b}_{\text{net}}, \end{aligned} \quad (1.16)$$

where $\mathbf{W}_{\text{net}} = \mathbf{W}_1 \mathbf{W}_2 \in \mathbb{R}^{n \times k}$ and $\mathbf{b}_{\text{net}} = \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2 \in \mathbb{R}^k$.

Proof by mathematical induction. Matrix multiplication is associative and closed under composition. The product of two matrices $\mathbf{W}_1 \in \mathbb{R}^{n \times m}$ and $\mathbf{W}_2 \in \mathbb{R}^{m \times k}$ is simply a single matrix $\mathbf{W}_{\text{net}} \in \mathbb{R}^{n \times k}$. Likewise, the affine transformation of bias vector $\mathbf{b}_1$ by $\mathbf{W}_2^\top$ added to $\mathbf{b}_2$ is simply a single constant vector $\mathbf{b}_{\text{net}} \in \mathbb{R}^k$.

By mathematical induction, for any cascade of L linear layers:

$$\mathbf{z} = \mathbf{W}_L^\top (\dots (\mathbf{W}_2^\top (\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) \dots) + \mathbf{b}_L = \mathbf{W}_{\text{net}}^\top \mathbf{x} + \mathbf{b}_{\text{net}}, \quad (1.17)$$

where:

$$\mathbf{W}_{\text{net}} = \prod_{l=1}^L \mathbf{W}_l \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}, \quad \mathbf{b}_{\text{net}} = \mathbf{b}_L + \sum_{l=1}^{L-1} \left(\prod_{j=l+1}^L \mathbf{W}_j \right)^\top \mathbf{b}_l. \quad (1.18)$$

The composite mapping is strictly linear. Stacking 100 purely linear layers introduces millions of redundant matrix multiplications while learning nothing beyond standard linear regression. It strictly cannot compute XOR or any non-linear function. ■

The fundamental takeaway. Non-linear activation functions are strictly and mathematically mandatory. Without non-linearity between layers, deep networks collapse into trivial shallow affine mappings. Non-linear activations warp and fold coordinate space, transforming linearly inseparable data manifolds into representations where classes can be cleanly separated.

1.2.3 Universal approximation theorem and bump-function synthesis

Given that non-linear activation functions prevent linear collapse, what family of functions can a non-linear neural network represent? The theoretical foundation validating neural networks as general-purpose function approximators is the *Universal Approximation Theorem*.

Theorem 1.2: Universal approximation theorem

Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be any continuous, non-constant, bounded, and monotonically increasing activation function (e.g., sigmoid) [?]. Let $I_n \subset \mathbb{R}^n$ denote a compact (closed and bounded) subset of n -dimensional Euclidean space, and let $C(I_n)$ denote the space of continuous real-valued functions on I_n . Then, for any target continuous function $g \in C(I_n)$ and any arbitrary error tolerance $\epsilon > 0$, there exist a finite integer $M \in \mathbb{N}$, parameter weights $\mathbf{w}_j \in \mathbb{R}^n$, outer weights $\alpha_j \in \mathbb{R}$, and biases $b_j \in \mathbb{R}$ such that the single-hidden-layer feedforward network:

$$F(\mathbf{x}) = \sum_{j=1}^M \alpha_j \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j) \quad (1.19)$$

satisfies uniform approximation error:

$$\sup_{\mathbf{x} \in I_n} |F(\mathbf{x}) - g(\mathbf{x})| < \epsilon. \quad (1.20)$$

Originally proved by George Cybenko [?] for sigmoidal activations and generalized by Kurt Hornik et al. [?] and Moshe Leshno et al. [?] to arbitrary continuous non-polynomial activations, this theorem guarantees that a feedforward network with a single hidden layer has the mathematical capacity to approximate any continuous function.

Intuitive geometric derivation: bump-function synthesis. To understand intuitively why a single hidden layer can approximate any continuous curve, consider a 1D function $g(x) : [a, b] \rightarrow \mathbb{R}$.

Step 1: Constructing a localized rectangular bump. Consider two sigmoid neurons with very high slope $w \gg 0$ located at spatial offsets x_1 and x_2 (with $x_1 < x_2$). As $w \rightarrow \infty$, the sigmoid $\sigma(w(x - x_1))$ converges to a Heaviside step function $H(x - x_1)$.

Subtracting the second sigmoid from the first produces a localized *bump function* (or rectangular wavelet):

$$B(x) = \sigma(w(x - x_1)) - \sigma(w(x - x_2)) \approx \begin{cases} 1 & \text{if } x_1 \leq x \leq x_2 \\ 0 & \text{otherwise} \end{cases}. \quad (1.21)$$

Figure ??(a) illustrates how two opposing sigmoids subtract to create a localized rectangular pulse.

Step 2: Riemann sum function reconstruction. By the definition of continuous functions on compact intervals, $g(x)$ can be approximated to arbitrary accuracy ϵ by a piecewise-constant step function consisting of M uniform sub-intervals $[x_k, x_{k+1}]$ with heights $c_k = g(x_k)$, exactly as in standard Riemann integration:

$$g(x) \approx \sum_{k=1}^M c_k B_k(x) = \sum_{k=1}^M c_k [\sigma(w(x - x_k)) - \sigma(w(x - x_{k+1}))]. \quad (1.22)$$

Each bump $B_k(x)$ requires exactly two hidden sigmoid neurons. Consequently, a single hidden layer containing $2M$ neurons can synthesize any continuous function to arbitrary precision, as visualized in Figure ??(b).

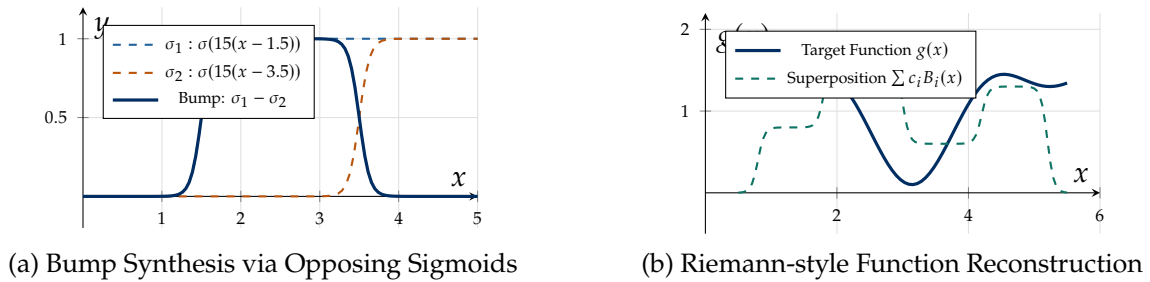


Figure 1.5: Visual derivation of the Universal Approximation Theorem. (a) Subtracting two shifted steep sigmoids produces a localized rectangular bump function $B(x)$; (b) Superimposing multiple scaled bump functions reconstructs any continuous target function to arbitrary accuracy ϵ .

1.2.4 Why deep networks if shallow networks are universal approximators?

If a single hidden layer can approximate any function, why does modern deep learning exclusively use deep architectures with dozens or hundreds of stacked layers (e.g., 96 layers in GPT-3)?

1. **Non-Constructive Existence:** The theorem guarantees that parameters exist, but provides no gradient-based algorithm to discover them and establishes no finite bound on the width M .
2. **Exponential Parameter Inefficiency:** Approximating highly compositional functions in n dimensions with a single hidden layer requires a width M that scales exponentially with input dimensionality ($M \sim O(2^n)$).
3. **Hierarchical Compositionality:** Deep networks factorize functions into hierarchical abstractions: early layers learn low-level primitives (subwords, syntax), intermediate layers learn grammatical structure, and deep layers capture long-range semantic discourse. Deep networks achieve equal approximation power with exponentially fewer parameters than shallow networks.

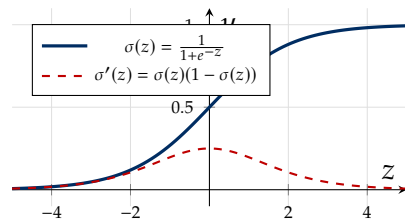
1.3 Taxonomy of neural activation functions

Modern neural NLP architectures deploy diverse activation functions tailored to hidden representations and output heads. Figure ?? presents exact mathematical plots and derivatives for the primary activation functions.

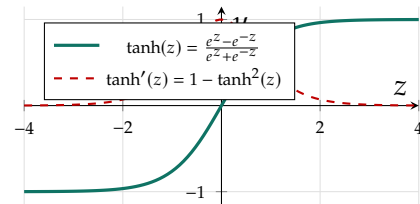
1. The logistic sigmoid (σ).

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{Range: } (0, 1), \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)). \quad (1.23)$$

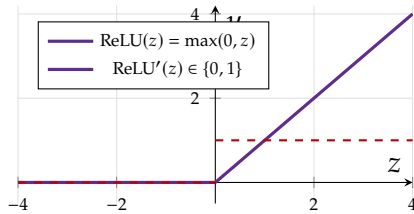
The derivative attains its global maximum of $\sigma'(0) = 0.25$ at the origin. For large positive or negative inputs ($|z| \geq 4$), $\sigma'(z) \rightarrow 0$, causing severe *vanishing gradients* in deep networks.



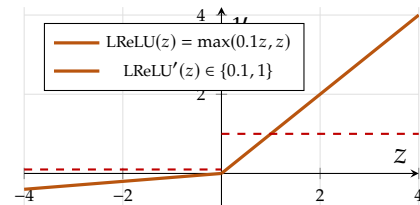
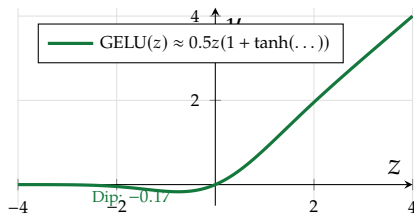
(a) Sigmoid (Logistic)



(b) Hyperbolic Tangent (tanh)



(c) Rectified Linear Unit (ReLU)

(d) Leaky ReLU ($\alpha = 0.1$)

(e) Gaussian Error Linear Unit (GELU)

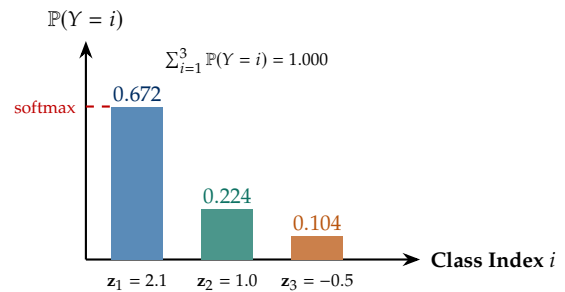
(f) Softmax Simplex Mapping ($\mathbb{R}^3 \rightarrow \Delta^2$)

Figure 1.6: Taxonomy of primary neural activation functions plotted across their operating ranges: (a) Sigmoid $\sigma(z)$ with peak derivative 0.25; (b) Zero-centered $\tanh(z)$ with peak derivative 1.0; (c) Piecewise linear $\text{ReLU}(z)$; (d) Leaky ReLU preventing dead units; (e) Non-monotonic $\text{GELU}(z)$ used in modern Transformers; (f) Multi-class Softmax mapping logits to the probability simplex.

2. Hyperbolic tangent (tanh).

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1, \quad \text{Range: } (-1, 1), \quad \tanh'(z) = 1 - \tanh^2(z). \quad (1.24)$$

The hyperbolic tangent is strictly zero-centered, possessing average activation values near zero, which accelerates gradient descent compared to non-negative sigmoids. However, it still saturates for $|z| \geq 3$, yielding vanishing gradients.

3. Rectified linear unit (ReLU).

$$\text{ReLU}(z) = \max(0, z), \quad \text{Range: } [0, \infty), \quad \text{ReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}. \quad (1.25)$$

ReLU completely eliminates gradient vanishing for positive pre-activations ($z > 0$). However, if a large gradient update shifts a neuron's weights such that $z \leq 0$ across all training samples, its activation and derivative permanently collapse to zero ($\text{ReLU}'(z) = 0$), a pathology known as the *dying ReLU problem*.

4. Leaky ReLU (LReLU).

$$\text{LReLU}(z) = \max(\alpha z, z) \quad \text{with } 0 < \alpha \ll 1, \quad \text{Range: } (-\infty, \infty), \quad \text{LReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ \alpha & \text{if } z \leq 0 \end{cases}. \quad (1.26)$$

By maintaining a small constant slope α (typically $\alpha = 0.01$) in the negative half-plane, Leaky ReLU guarantees that a non-zero gradient always backpropagates, preventing dead neurons.

5. Gaussian error linear unit (GELU). Introduced by Dan Hendrycks and Kevin Gimpel, GELU scales the input by its cumulative standard normal probability $\Phi(z)$:

$$\text{GELU}(z) = z \cdot \Phi(z) = z \cdot \frac{1}{2} \left[1 + \text{erf} \left(\frac{z}{\sqrt{2}} \right) \right] \approx 0.5z \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (z + 0.044715z^3) \right) \right). \quad (1.27)$$

GELU is smooth, differentiable everywhere, and non-monotonic, dipping to a global minimum of ≈ -0.17 at $z \approx -0.75$. This smoothness provides curvature information during optimization, making GELU the default activation in Transformer language models (GPT-2, GPT-3, BERT, RoBERTa).

6. Softmax. For categorical multi-class predictions over C mutually exclusive classes, Softmax normalizes a real-valued logit vector $\mathbf{z} = [z_1, \dots, z_C]^\top \in \mathbb{R}^C$ onto the unit probability simplex Δ^{C-1} :

$$\hat{y}_i = \text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \quad \text{such that } \hat{y}_i \in (0, 1) \quad \text{and} \quad \sum_{i=1}^C \hat{y}_i = 1. \quad (1.28)$$

Softmax forms the standard vocabulary output head of modern autoregressive language models, computing next-token probability distributions across vocabulary $\mathcal{V}$.

Table ?? provides a comparative reference summarizing the mathematical properties of each activation function.

Table 1.1: Comprehensive taxonomy of neural activation functions, mathematical formulations, operating bounds, and architectural domains.

Activation	Domain	Range	Peak Gradient	Primary Use Case in Language Modeling
Sigmoid (σ)	$\mathbb{R}$	$(0, 1)$	$0.25 (z = 0)$	Binary classification output heads; gating mechanisms in LSTM/GRU highways.
Tanh	$\mathbb{R}$	$(-1, 1)$	$1.00 (z = 0)$	Recurrent hidden state candidate generation; zero-centered coordinate representations.
ReLU	$\mathbb{R}$	$[0, \infty)$	$1.00 (z > 0)$	Classical deep feedforward networks; compute-efficient linear regions.
Leaky ReLU	$\mathbb{R}$	$\mathbb{R}$	$1.00 (z > 0), \alpha (z \leq 0)$	Prevents dying neuron syndrome; robust discriminator networks.
GELU	$\mathbb{R}$	$[-0.17, \infty)$	≈ 1.08	Canonical Transformer activation (GPT, BERT); smooth, probabilistic gating.
Softmax	$\mathbb{R}^C$	Δ^{C-1}	Matrix Jacobian	Next-token prediction vocabulary head; maps logits to discrete probability simplex.
Identity	$\mathbb{R}$	$\mathbb{R}$	$1.00 (\forall z)$	Regression output head; unbounded linear prediction.

1.4 Architectural task formulations and software engineering case studies

In machine learning applications for software engineering, task formulations map specific software artifacts (commits, ASTs, bug reports, source code tokens) into continuous predictions. Three primary architectural heads dominate:

1.4.1 1. Binary classification: automated vulnerability detection

Consider predicting whether a proposed code commit contains a security vulnerability (e.g., buffer overflow, SQL injection).

- **Input Vector:** $\mathbf{x} \in \mathbb{R}^d$ (extracted code embeddings or static analysis metrics).
- **Output Layer:** A single linear neuron producing logit $z = \mathbf{w}^\top \mathbf{h} + b \in \mathbb{R}$.
- **Activation:** Logistic Sigmoid $\hat{y} = \sigma(z) = \mathbb{P}(\text{Vulnerable} = 1 \mid \mathbf{x}) \in [0, 1]$.
- **Decision Rule:** Flag for manual audit if $\hat{y} \geq \tau$ (where $\tau = 0.5$ by default, tuned to optimize recall).

Figure ??(a) illustrates the resulting separating hyperplane in feature space.

1.4.2 2. Multi-class classification: automated AST node type prediction

In program repair and neural compiler optimization, models classify Abstract Syntax Tree (AST) nodes into C discrete syntactic types (e.g., MethodDeclaration, VariableDeclaration, IfStatement, WhileStatement).

- **Output Layer:** C parallel linear neurons producing logit vector $\mathbf{z} = \mathbf{W}^\top \mathbf{h} + \mathbf{b} \in \mathbb{R}^C$.
- **Activation:** Softmax operator $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}) \in \Delta^{C-1}$.

- **Decision Rule:** Predicted AST type $\hat{c} = \arg \max_i \hat{y}_i$.

Figure ??(b) diagrams the resulting Voronoi-like partitioned multi-class decision space.

1.4.3 3. Regression: pull request review effort estimation

Engineering managers must predict continuous quantities, such as the estimated review time (in hours) required for an incoming pull request based on code churn, changed files, and author experience.

- **Output Layer:** A single linear neuron producing scalar $z = \mathbf{w}^\top \mathbf{h} + b$.
- **Activation:** Linear Identity function $f(z) = z \in \mathbb{R}$.
- **Prediction:** $\hat{y} = z \in \mathbb{R}$ (continuous unbounded prediction).

Figure ??(c) illustrates continuous line fitting through software engineering observation points.

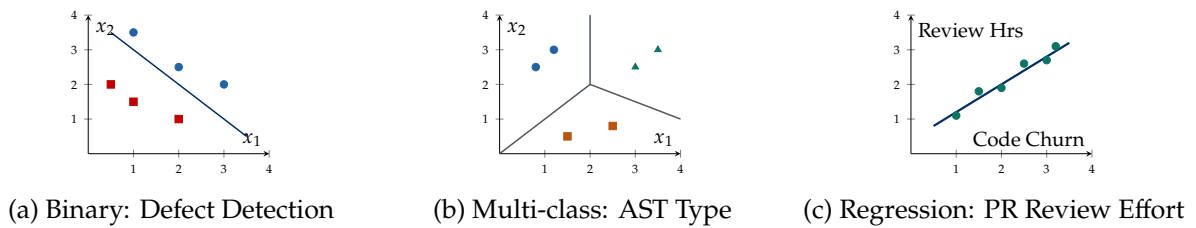


Figure 1.7: Task formulations across Software Engineering workflows: (a) Binary vulnerability detection with separating hyperplane; (b) Multi-class defect categorization with partitioned decision regions; (c) Continuous PR review effort regression with residual errors.

1.5 Loss function formulations and optimization landscapes

1.5.1 Loss function formulations

Training a neural network requires an objective function $\mathcal{L}(\theta)$ that measures the discrepancy between model predictions $\hat{y}$ and ground-truth targets y .

1. **Mean squared error (MSE).** For continuous regression targets:

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2. \quad (1.29)$$

2. **Binary cross-entropy (BCE).** For binary classification with target $y \in \{0, 1\}$ and predicted probability $\hat{y} = \sigma(z) \in [0, 1]$:

$$\mathcal{L}_{\text{BCE}}(\theta) = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y}). \quad (1.30)$$

3. **Categorical cross-entropy (CCE).** For multi-class classification over C mutually exclusive classes with one-hot target vector $\mathbf{y} \in \{0, 1\}^C$:

$$\mathcal{L}_{\text{CCE}}(\theta) = - \sum_{i=1}^C y_i \ln(\hat{y}_i) = - \ln(\hat{y}_{\text{target}}) = - \ln \left(\frac{e^{z_{\text{target}}}}{\sum_{j=1}^C e^{z_j}} \right). \quad (1.31)$$

Figure ?? compares the mathematical profiles of MSE and BCE for a positive target $y = 1$.

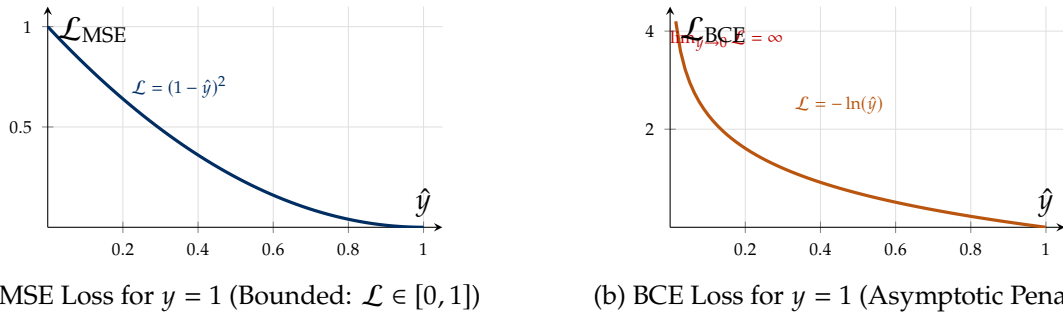


Figure 1.8: Comparison of loss function behaviors for target $y = 1$. While MSE is bounded quadratically at 1.0, BCE imposes an asymptotic infinite penalty ($\mathcal{L} \rightarrow \infty$) as predicted probability $\hat{y} \rightarrow 0$, providing steep, non-vanishing corrective gradients.

1.5.2 Rigorous proof: why MSE fails for classification

A fundamental theoretical question is: *Why do we never use Mean Squared Error with sigmoid activation for classification?* The answer lies in gradient saturation and non-convexity.

Theorem 1.3: Gradient saturation of MSE under sigmoid activation

Let a binary classification model predict $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$ with ground-truth target $y \in \{0, 1\}$. Under the quadratic loss $\mathcal{L}_{\text{MSE}} = \frac{1}{2}(y - \hat{y})^2$, the gradient with respect to the pre-activation logit z is:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}). \quad (1.32)$$

Conversely, under Binary Cross-Entropy $\mathcal{L}_{\text{BCE}} = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$, the gradient is:

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} = \hat{y} - y. \quad (1.33)$$

Proof. 1. For MSE, applying the calculus chain rule:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = \frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z}. \quad (1.34)$$

The derivative of the loss with respect to $\hat{y}$ is:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} \left[\frac{1}{2}(y - \hat{y})^2 \right] = -(y - \hat{y}).$$

The derivative of the sigmoid is $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = \hat{y}(1 - \hat{y})$. Substituting:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -(y - \hat{y})\hat{y}(1 - \hat{y}). \quad (1.35)$$

2. For BCE, expanding the loss:

$$\mathcal{L}_{\text{BCE}} = -y \ln(\sigma(z)) - (1 - y) \ln(1 - \sigma(z)). \quad (1.36)$$

Differentiating directly with respect to logit z :

$$\begin{aligned}
 \frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} &= -y \frac{\sigma'(z)}{\sigma(z)} - (1-y) \frac{-\sigma'(z)}{1-\sigma(z)} \\
 &= -y \frac{\sigma(z)(1-\sigma(z))}{\sigma(z)} + (1-y) \frac{\sigma(z)(1-\sigma(z))}{1-\sigma(z)} \\
 &= -y(1-\sigma(z)) + (1-y)\sigma(z) \\
 &= -y + y\sigma(z) + \sigma(z) - y\sigma(z) \\
 &= \sigma(z) - y = \hat{y} - y. \quad \blacksquare
 \end{aligned}$$

Analytical comparison of optimization behavior. The fundamental architectural flaws of MSE for classification are immediately apparent from Equations (??) and (??):

- **The Worst-Case MSE Pathology (Confident and Wrong):** Suppose the true label is $y = 1$, but the network is completely mistaken, predicting $z = -10 \implies \hat{y} \approx 0.000045$. The prediction error is maximal: $(y - \hat{y}) \approx 1.0$. However, evaluating the MSE gradient:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -(1.0) \times (0.000045) \times (0.999955) \approx -0.000045 \approx 0!$$

The gradient completely vanishes! Even though the model is entirely wrong, parameter updates halt because the sigmoid derivative $\hat{y}(1 - \hat{y})$ saturates to zero. Optimization gets trapped on a flat plateau.

- **The BCE Remedy:** Under BCE, for the identical scenario ($y = 1, \hat{y} \approx 0$):

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} = \hat{y} - y = 0 - 1 = -1.0.$$

The sigmoid derivative in the numerator cancels perfectly with the denominator in the derivative of the logarithm! The gradient is strictly linear with respect to the error $(y - \hat{y})$ and provides maximum learning signal precisely when the model is most wrong.

- **Non-Convexity:** While MSE with linear regression is strictly convex, combining MSE with non-linear sigmoid activations creates a non-convex loss surface with numerous local plateaus. In contrast, BCE is strictly convex with respect to logit z .

1.5.3 Maximum likelihood estimation derivation of cross-entropy

Cross-entropy is not an empirical heuristic; it is the mathematically canonical loss derived from Maximum Likelihood Estimation (MLE).

Bernoulli likelihood derivation (BCE). For binary classification, target $y_i \in \{0, 1\}$ follows a Bernoulli distribution parameterized by probability $p_i = \hat{y}_i = \sigma(z_i)$:

$$\mathbb{P}(Y = y_i \mid \mathbf{x}_i) = \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}. \quad (1.37)$$

For an independent dataset of N samples, the total likelihood is $\mathcal{L}_{\text{data}}(\theta) = \prod_{i=1}^N \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$. Taking the negative log-likelihood (NLL):

$$\mathcal{L}(\theta) = -\ln \mathcal{L}_{\text{data}}(\theta) = -\sum_{i=1}^N [y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i)] = \sum_{i=1}^N \mathcal{L}_{\text{BCE},i}. \quad (1.38)$$

Minimizing BCE is mathematically identical to maximizing the data likelihood under a Bernoulli distribution.

Categorical likelihood derivation (CCE). For multi-class classification over C classes with one-hot targets $\mathbf{y}_i$, the likelihood follows a Categorical distribution:

$$\mathbb{P}(\mathbf{y}_i | \mathbf{x}_i) = \prod_{c=1}^C \hat{y}_{ic}^{y_{ic}} \implies -\ln \mathbb{P}(\mathbf{y}_i | \mathbf{x}_i) = -\sum_{c=1}^C y_{ic} \ln \hat{y}_{ic} = \mathcal{L}_{\text{CE}}. \quad (1.39)$$

1.5.4 Gradient descent and adaptive optimizers

Parameters θ are updated iteratively along the negative gradient:

$$\theta^{(t+1)} := \theta^{(t)} - \alpha \nabla_{\theta} \mathcal{L}(\theta^{(t)}), \quad (1.40)$$

where $\alpha > 0$ is the learning rate.

Limitations of vanilla gradient descent. In deep neural architectures, vanilla Gradient Descent struggles with:

1. **Ravines and Ill-Conditioned Curvature:** Valleys where gradient magnitude is large across the ravine walls but tiny along the base, causing erratic cross-valley oscillation.
2. **Saddle Points:** Points where $\nabla \mathcal{L} = 0$ while the Hessian possesses both positive and negative eigenvalues.

Adaptive remedies: Momentum and Adam.

- **Momentum:** Incorporates an exponentially decaying velocity vector $\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta) \nabla \mathcal{L}(\theta_t)$, accelerating descent through saddle regions.
- **Adam (Adaptive Moment Estimation) [?]:** Tracks running averages of both gradients (first moment $\mathbf{m}_t$) and squared gradients (second uncentered moment $\mathbf{v}_t$), with bias corrections:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad \hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad (1.41)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \quad (1.42)$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \hat{\mathbf{m}}_t. \quad (1.43)$$

Adam scales learning rates adaptively per parameter, making it the standard optimizer across LLM pre-training and fine-tuning pipelines.

1.6 Computational graphs and exact adjoint backpropagation

To compute gradients efficiently across billions of parameters, deep learning frameworks (PyTorch, JAX) represent mathematical workflows as a *Directed Acyclic Graph (DAG)*, known as a *computational graph*.

Definition 1.2: Computational graph

A computational graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a directed acyclic graph where:

- Each node $v \in \mathcal{V}$ represents an input variable, parameter, or primitive mathematical operation ($+$, $\times$, $-$, $\exp$).

- Each directed edge $e = (u, v) \in \mathcal{E}$ represents intermediate data flow from variable u into operation v .

1.6.1 Exact backpropagation walkthrough from slide 24

We reproduce and derive the exact pedagogical computational graph and adjoint trace from Slide 24 of the course materials.

Problem setup. Consider a model with two scalar parameters $\theta_1, \theta_2 \in \mathbb{R}$ evaluated on training pair $(x, y) \in \mathbb{R}^2$:

$$\text{Model Prediction: } \hat{y} = \theta_1 \theta_2 x, \quad (1.44)$$

$$\text{Objective Loss: } \mathcal{L} = (\theta_1 \theta_2 x - y)^2. \quad (1.45)$$

Forward decomposition. The graph decomposes Equation (??) into four primitive operations:

$$a = \theta_1 \theta_2 \quad (\text{parameter interaction product}), \quad (1.46)$$

$$b = ax = \theta_1 \theta_2 x \quad (\text{prediction } \hat{y}), \quad (1.47)$$

$$c = b - y = \theta_1 \theta_2 x - y \quad (\text{residual error}), \quad (1.48)$$

$$\mathcal{L} = c^2 = (\theta_1 \theta_2 x - y)^2 \quad (\text{scalar squared loss}). \quad (1.49)$$

Backward adjoint chain rule derivation. Backpropagation computes the derivative of scalar loss $\mathcal{L}$ with respect to every intermediate variable and parameter by traversing the graph in reverse topological order:

$$\text{Step 1: } \frac{\partial \mathcal{L}}{\partial c} = \frac{\partial(c^2)}{\partial c} = 2c. \quad (1.50)$$

$$\text{Step 2: } \frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial c} \frac{\partial c}{\partial b} = 2c \cdot \frac{\partial(b - y)}{\partial b} = 2c \cdot 1 = 2c. \quad (1.51)$$

$$\text{Step 3: } \frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial b} \frac{\partial b}{\partial a} = 2c \cdot \frac{\partial(ax)}{\partial a} = 2c \cdot x = 2cx. \quad (1.52)$$

$$\text{Step 4: } \frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_1} = (2cx) \cdot \frac{\partial(\theta_1 \theta_2)}{\partial \theta_1} = 2cx \cdot \theta_2 = 2(\theta_1 \theta_2 x - y)x \theta_2. \quad (1.53)$$

$$\text{Step 5: } \frac{\partial \mathcal{L}}{\partial \theta_2} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_2} = (2cx) \cdot \frac{\partial(\theta_1 \theta_2)}{\partial \theta_2} = 2cx \cdot \theta_1 = 2(\theta_1 \theta_2 x - y)x \theta_1. \quad (1.54)$$

Figure ?? illustrates this complete forward and backward adjoint execution.

💡 Intuition & analogy: 3Blue1Brown's geometric view of backpropagation

Grant Sanderson (3Blue1Brown) articulates backpropagation not as abstract calculus algebra, but as a systematic tracking of sensitivity *nudges*. Imagine wiggling parameter θ_1 by an infinitesimal perturbation ϵ . That nudge ripples through the product node to scale a by $\epsilon \theta_2$. The change in a wiggles b by $\epsilon \theta_2 x$, which directly shifts error c and finally scales loss $\mathcal{L}$ by $2c \epsilon \theta_2 x$. Backpropagation simply reads these connected multiplicative sensitivity tracks in reverse, gathering the composite derivative at each fork in the graph.

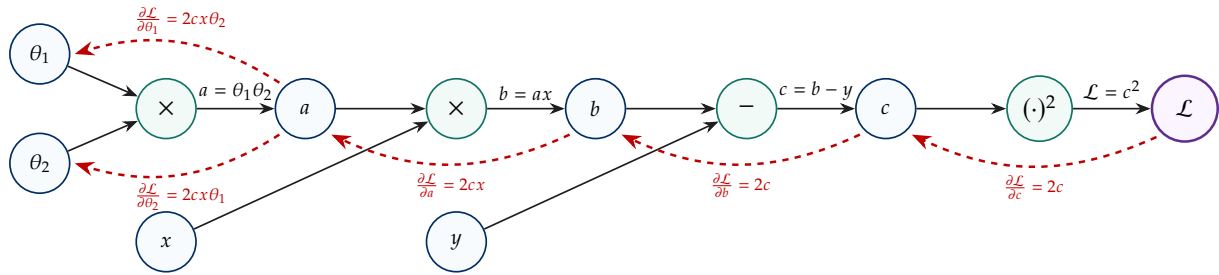


Figure 1.9: Directed Acyclic Computational Graph (DAG) for model $\hat{y} = \theta_1 \theta_2 x$ with quadratic loss $\mathcal{L} = (\theta_1 \theta_2 x - y)^2$ (reproduced from Slide 24). Solid black arrows trace forward execution; dashed red arrows trace reverse-mode adjoint backpropagation via the chain rule [?].

External deep dive: Building autograd from scratch: Andrej Karpathy's Micrograd

Andrej Karpathy's open-source library *Micrograd* implements reverse-mode automatic differentiation in fewer than 100 lines of pure Python, demonstrating the exact computational graph mechanics shown in Figure ??.

Each scalar variable is wrapped inside a `Value` class containing:

- `.data`: the floating-point scalar computed during the forward pass.
- `.grad`: the accumulated adjoint derivative $\partial \mathcal{L} / \partial \cdot$, initialized to 0.0.
- `._prev`: a set storing pointers to operand children nodes.
- `._backward`: a closure storing the local multivariate chain rule.

For example, multiplication is implemented as:

```
1 def __mul__(self, other):
2     other = other if isinstance(other, Value) else Value(other)
3     out = Value(self.data * other.data, (self, other), '*')
4     def _backward():
5         self.grad += other.data * out.grad
6         other.grad += self.data * out.grad
7     out._backward = _backward
8     return out
```

When calling `loss.backward()`, the engine performs a **topological sort** on the DAG to establish reverse dependency ordering, sets `loss.grad = 1.0`, and executes `node._backward()` across all parent nodes in reverse topological order.

1.6.2 Vectorized matrix adjoint backpropagation for dense layers

While scalar computational graphs illustrate the chain rule clearly, production deep learning libraries execute backpropagation via *vectorized matrix calculus* across high-dimensional tensors. We now derive the exact vectorized matrix adjoint equations for fully connected layers with explicit tensor dimensions.

Layer formulation and forward variables. Consider a fully connected dense layer mapping an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ through weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and bias $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$ to pre-activation

$\mathbf{z} \in \mathbb{R}^{d_{\text{out}}}$ and activation $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$:

$$\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{b} \in \mathbb{R}^{d_{\text{out}}}, \quad (1.55)$$

$$\mathbf{y} = f(\mathbf{z}) \in \mathbb{R}^{d_{\text{out}}}, \quad (1.56)$$

where scalar component z_j expands as:

$$z_j = \sum_{i=1}^{d_{\text{in}}} W_{ij} x_i + b_j. \quad (1.57)$$

The layer output $\mathbf{y}$ propagates downstream to contribute to a final scalar objective loss $\mathcal{L} \in \mathbb{R}$.

Adjoint definition. Assume downstream backpropagation has computed the loss gradient with respect to layer pre-activation $\mathbf{z}$. We define the adjoint error vector $\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$:

$$\boldsymbol{\delta} \triangleq \frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \odot f'(\mathbf{z}) \in \mathbb{R}^{d_{\text{out}}}, \quad (1.58)$$

where $\odot$ denotes the elementwise Hadamard product.

1. Weight matrix adjoint derivation ($\frac{\partial \mathcal{L}}{\partial \mathbf{W}}$). To compute the derivative of scalar loss $\mathcal{L}$ with respect to individual matrix element W_{ij} , we apply the multivariate chain rule across all output pre-activations z_k :

$$\frac{\partial \mathcal{L}}{\partial W_{ij}} = \sum_{k=1}^{d_{\text{out}}} \frac{\partial \mathcal{L}}{\partial z_k} \frac{\partial z_k}{\partial W_{ij}}. \quad (1.59)$$

Because $z_k = \sum_{l=1}^{d_{\text{in}}} W_{lk} x_l + b_k$, the partial derivative is non-zero only when $k = j$:

$$\frac{\partial z_k}{\partial W_{ij}} = \begin{cases} x_i & \text{if } k = j \\ 0 & \text{if } k \neq j \end{cases}. \quad (1.60)$$

Substituting this back:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}} = \frac{\partial \mathcal{L}}{\partial z_j} x_i = \delta_j x_i = x_i \delta_j. \quad (1.61)$$

Collecting all elements W_{ij} into matrix form reveals that the weight gradient is identically the **outer product** of the upstream input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ and the adjoint error vector $\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{x} \boldsymbol{\delta}^\top \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}. \quad (1.62)$$

2. Bias vector adjoint derivation ($\frac{\partial \mathcal{L}}{\partial \mathbf{b}}$). For each scalar bias component b_j :

$$\frac{\partial \mathcal{L}}{\partial b_j} = \sum_{k=1}^{d_{\text{out}}} \frac{\partial \mathcal{L}}{\partial z_k} \frac{\partial z_k}{\partial b_j} = \delta_j \cdot 1 = \delta_j. \quad (1.63)$$

In vector notation:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}. \quad (1.64)$$

3. Upstream input adjoint derivation ($\frac{\partial \mathcal{L}}{\partial \mathbf{x}}$). To propagate error signals back to previous layers in a deep neural network, we must evaluate the gradient of loss $\mathcal{L}$ with respect to the input feature vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$. Applying the chain rule to input component x_i :

$$\frac{\partial \mathcal{L}}{\partial x_i} = \sum_{k=1}^{d_{\text{out}}} \frac{\partial \mathcal{L}}{\partial z_k} \frac{\partial z_k}{\partial x_i} = \sum_{k=1}^{d_{\text{out}}} \delta_k W_{ik} = \sum_{k=1}^{d_{\text{out}}} W_{ik} \delta_k. \quad (1.65)$$

Recognizing this sum as the i -th element of the matrix-vector multiplication between weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and adjoint vector $\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{W} \boldsymbol{\delta} \in \mathbb{R}^{d_{\text{in}}}. \quad (1.66)$$

Figure ?? diagrams this vectorized adjoint error propagation across the dense layer.

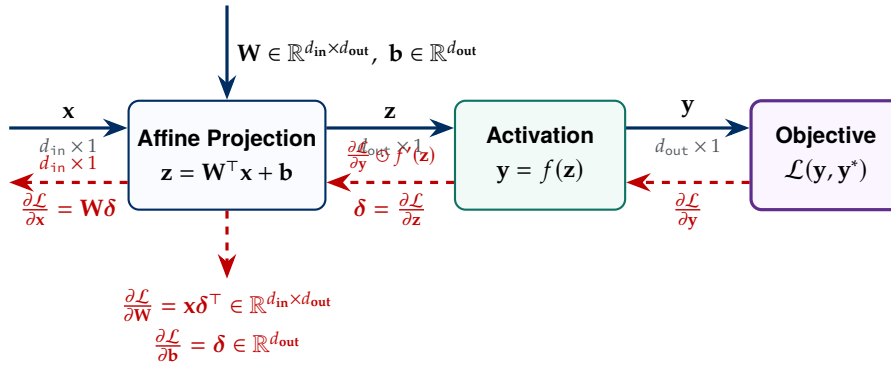


Figure 1.10: Vectorized matrix adjoint backpropagation across a fully connected dense layer. Solid navy arrows trace forward execution mapping input $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ to output $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$; dashed red arrows trace reverse-mode adjoint error propagation ($\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$), yielding parameter updates $\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{x} \boldsymbol{\delta}^T$, $\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \boldsymbol{\delta}$, and upstream gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{W} \boldsymbol{\delta}$.

Mini-batch matrix generalization. In GPU implementations, B samples are grouped into a batch data matrix $\mathbf{X} \in \mathbb{R}^{B \times d_{\text{in}}}$. The layer computes:

$$\mathbf{Z} = \mathbf{XW} + \mathbf{1}_B \mathbf{b}^T \in \mathbb{R}^{B \times d_{\text{out}}}, \quad \mathbf{Y} = f(\mathbf{Z}) \in \mathbb{R}^{B \times d_{\text{out}}}. \quad (1.67)$$

Defining batch adjoint matrix $\boldsymbol{\Delta} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}} \in \mathbb{R}^{B \times d_{\text{out}}}$, the gradient equations generalize to highly efficient Basic Linear Algebra Subprograms (BLAS) matrix multiplications:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{X}^T \boldsymbol{\Delta} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}, \quad (1.68)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \boldsymbol{\Delta}^T \mathbf{1}_B \in \mathbb{R}^{d_{\text{out}}}, \quad (1.69)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \boldsymbol{\Delta} \mathbf{W}^T \in \mathbb{R}^{B \times d_{\text{in}}}. \quad (1.70)$$

These matrix equations form the computational workhorse executed across billions of parameters in modern GPU training clusters.

1.7 The continuous-discrete representation impasse: bridge to word embeddings

In this chapter, we have developed the atomic foundations of deep learning:

1. **Continuous Units:** The artificial perceptron computes weighted inner products over real-valued vectors.
2. **Non-Linearity:** Non-linear activations (σ , $\tanh$, ReLU , GELU) prevent multi-layer linear stacking collapse, folding space to solve linearly inseparable problems like XOR and enabling universal function approximation.
3. **Principled Optimization:** Cross-entropy objective functions provide linear error signals without gradient saturation, and backpropagation calculates matrix adjoints across computational DAGs via vectorized matrix calculus.

The representation impasse. However, applying this continuous machinery to natural language and software source code exposes an immediate, fundamental impasse:

Deep neural networks are **differentiable continuous operators**: $f(\mathbf{x}; \theta) = \mathbf{W}^\top \mathbf{x} + \mathbf{b}$ ($\mathbf{x} \in \mathbb{R}^d$).

In stark contrast, human natural language and software source code are sequences of **discrete, categorical text symbols**:

$$W = (w_1, w_2, \dots, w_N), \quad w_t \in \mathcal{V} = \{\text{public}, \text{class}, \text{static}, \text{void}, \dots\}.$$

We cannot compute inner products $\mathbf{w}^\top \mathbf{x}$, calculate derivatives, or execute backpropagation on raw ASCII or Unicode strings.

Why naive numerical encodings fail. Two obvious baseline approaches fail catastrophically:

- **Scalar Integer Indexing:** Mapping each vocabulary word to a scalar integer ($w_t \in \{1, 2, \dots, |\mathcal{V}|\}$) imposes an arbitrary, fictitious metric:

$$\text{class} \rightarrow 42, \quad \text{interface} \rightarrow 43, \quad \text{int} \rightarrow 840.$$

This forces the model to assume that `interface` is geometrically adjacent to `class` ($|43 - 42| = 1$), while `int` is vastly distant ($|840 - 42| = 798$), corrupting optimization.

- **Local One-Hot Encodings:** Representing each word as an indicator vector $\mathbf{e}_i \in \{0, 1\}^{|\mathcal{V}|}$ creates sparse vectors of massive dimensionality ($|\mathcal{V}| \geq 50,000$). Crucially, standard basis vectors are mutually orthogonal:

$$\mathbf{e}_i^\top \mathbf{e}_j = 0 \quad \forall i \neq j, \quad d_E(\mathbf{e}_i, \mathbf{e}_j) = \sqrt{2}.$$

Every word is equidistant from every other word, providing zero semantic similarity, zero parameter sharing during backpropagation, and crippling data efficiency.

The path forward. To bridge this chasm between discrete symbolic languages and continuous neural operators, we must construct a dense, continuous, differentiable mapping:

$$\phi : \mathcal{V} \longrightarrow \mathbb{R}^d \quad (d \ll |\mathcal{V}|),$$

that maps discrete words into low-dimensional semantic vector spaces where geometric distance preserves syntactic and semantic relationships.

This challenge leads directly to Chapter ??: *Distributed Word Representations and Word2Vec*.